JSPM NARHE TECHNICAL CAMPUS

Rajarshi SHAHU SCHOOL OF ENGINEERING AND RESEARCH

Survey No. 12/2/2 & 14/9, Near Pune Bangalore Highway, Narhe, Tal- Haveli, Pune, Maharashtra-411041.

Department of AI & DS Engineering

A.Y. 2026-27

Class: T.E

Course Name: Machine Learning Lab

Course Code: PCC353AID

Lab Manual

Incharge: Prof. D. A. Shivatare

List of Experiments

Exp. No.	Title of Experiment
1	Feature Engineering: Handling Missing Values, Encoding Categorical Variables and Scaling Numerical Attributes
2	Dimensionality Reduction using PCA and LDA
3	Multiple Linear Regression and Model Evaluation using MSE, RMSE and R^2
4	Multiple Linear Regression: Predictive Capability Assessment on an Alternate Dataset
5	Regularization Techniques: Ridge and Lasso Regression
6	Binary Classification using Logistic Regression: Confusion Matrix, Precision, Recall, and F1-Score
7	Support Vector Machine Classification: Linear and Non-linear Kernels
8	K-Means Clustering with Optimal Cluster Selection using Elbow / Silhouette Method
9	Density-Based Clustering using DBSCAN
10	Ensemble Learning: Random Forest / Boosting vs Baseline Model

List of Tables

Table No.	Title of Table
1.1	Comparison Table — Feature Engineering: Handling Missing Values, Encoding Categorical Variables and Scaling Numerical Attributes
1.2	Observation Table — Feature Engineering: Handling Missing Values, Encoding Categorical Variables and Scaling Numerical Attributes
2.1	Comparison Table — Dimensionality Reduction using PCA and LDA
2.2	Observation Table — Dimensionality Reduction using PCA and LDA
3.1	Comparison Table — Multiple Linear Regression and Model Evaluation using MSE, RMSE and R^2
3.2	Observation Table — Multiple Linear Regression and Model Evaluation using MSE, RMSE and R^2
4.1	Comparison Table — Multiple Linear Regression: Predictive Capability Assessment on an Alternate Dataset
4.2	Observation Table — Multiple Linear Regression: Predictive Capability Assessment on an Alternate Dataset
5.1	Comparison Table — Regularization Techniques: Ridge and Lasso Regression
5.2	Observation Table — Regularization Techniques: Ridge and Lasso Regression
6.1	Comparison Table — Binary Classification using Logistic Regression: Confusion Matrix, Precision, Recall, and F1-Score
6.2	Observation Table — Binary Classification using Logistic Regression: Confusion Matrix, Precision, Recall, and F1-Score
7.1	Comparison Table — Support Vector Machine Classification: Linear and Non-linear Kernels
7.2	Observation Table — Support Vector Machine Classification: Linear and Non-linear Kernels
8.1	Comparison Table — K-Means Clustering with Optimal Cluster Selection using Elbow / Silhouette Method
8.2	Observation Table — K-Means Clustering with Optimal Cluster Selection using Elbow / Silhouette Method
9.1	Comparison Table — Density-Based Clustering using DBSCAN
9.2	Observation Table — Density-Based Clustering using DBSCAN
10.1	Comparison Table — Ensemble Learning: Random Forest / Boosting vs Baseline Model
10.2	Observation Table — Ensemble Learning: Random Forest / Boosting vs Baseline Model

Cb=Acknowledgement

I would like to express my sincere gratitude to my subject teacher and the Head of the Department for their invaluable guidance, continuous encouragement, and constant support throughout the completion of this Machine Learning Laboratory manual.

I am also thankful to the laboratory assistants and technical staff for providing the necessary infrastructure, software resources, and a conducive environment for carrying out the practical experiments included in this manual.

I extend my heartfelt thanks to my classmates and friends for their cooperation and helpful discussions during these experiments.

Finally, I am grateful to my institution for providing this opportunity to enhance my practical knowledge and skills in the field of Machine Learning

— Student Name

Experiment No. 1

Feature Engineering: Handling Missing Values, Encoding Categorical Variables and Scaling Numerical Attributes

---

1. Aim / Problem Statement

To apply basic feature engineering techniques such as handling missing values, encoding categorical variables, and scaling numerical attributes on a suitable open-access dataset in order to prepare a clean and model-ready dataset for machine learning analysis.

2. Objectives

- To understand the importance of data pre-processing in the machine learning pipeline.
- To identify and treat missing values using suitable imputation strategies.
- To convert categorical (non-numeric) attributes into a numeric representation using encoding techniques.
- To scale/normalize numerical attributes so that all features contribute equally to model learning.
- To produce a clean, consistent, model-ready dataset.

3. Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries: pandas, numpy, scikit-learn (sklearn.preprocessing, sklearn.impute), matplotlib, seaborn

4. Introduction

4.1 Concept

Feature engineering is the process of using domain knowledge and statistical techniques to transform raw data into features that better represent the underlying problem to the predictive models, resulting in improved model accuracy. Real-world datasets are rarely clean: they contain missing entries, a mix of numeric and categorical columns, and features on very different scales (e.g., "age" in years versus "salary" in rupees). Feeding such raw data directly into a machine learning algorithm can lead to biased, unstable, or completely invalid models, since most algorithms assume complete, purely numeric, and comparably-scaled input.

4.2 Background

Historically, statisticians handled missing data and scaling manually for small, curated datasets. As datasets grew larger and more heterogeneous (survey data, sensor data, transactional data), pre-processing became a formal stage of the Knowledge Discovery in Databases (KDD) and CRISP-DM process models, and dedicated libraries such as scikit-learn's `preprocessing` and `impute` modules were built to standardise these operations.

4.3 Applications

- Preparing patient records (with missing lab values) for disease-prediction models.
- Encoding categorical survey responses (gender, city, product category) for regression/classification.
- Scaling financial features (income, loan amount) before applying gradient-descent based algorithms.
- Preparing e-commerce clickstream data (mixed categorical/numeric) for recommendation systems.

4.4 Advantages

- Improves model accuracy and convergence speed.
- Reduces the influence of outliers and differently-scaled features.
- Allows algorithms that require numeric input (e.g., SVM, KNN, Neural Networks) to use categorical data.
- Produces a reusable, standardised pre-processing pipeline.

4.5 Limitations

- Imputation can introduce bias if the missingness is not random (MNAR data).
- One-hot encoding can cause the "curse of dimensionality" for high-cardinality categorical features.
- Over-scaling can sometimes remove meaningful magnitude information (e.g., in tree-based models scaling is unnecessary).

5. Theory

Data pre-processing for feature engineering typically involves three major sub-tasks that are performed in sequence: (1) Missing Value Treatment, (2) Categorical Encoding, and (3) Feature Scaling.

1. Missing Value Treatment: Missing values may occur as NaN/blank entries and are broadly classified as MCAR (Missing Completely at Random), MAR (Missing at Random), and MNAR (Missing Not at Random). Common strategies include: deletion (row/column drop when missingness is small), mean/median imputation for numeric columns, mode imputation for categorical columns, and advanced techniques such as KNN-Imputer or Iterative Imputer (regression-based).

2. Categorical Encoding: Machine learning algorithms operate on numbers, so categorical (text/label) columns must be converted:

• Label Encoding — assigns an integer to each category (0,1,2,...); suitable for ordinal data.

• One-Hot Encoding — creates a new binary column for every category; suitable for nominal data with low cardinality.

• Ordinal Encoding — similar to label encoding but respects a defined order.

3. Feature Scaling: Numeric attributes are transformed onto a common scale:

• Standardization (Z-score): $x' = (x - \mu) / \sigma$, resulting in zero mean and unit variance.

- Min-Max Normalization: x' = (x − min) / (max − min), scaling to a [0,1] range.
- Robust Scaling: uses median and interquartile range, useful with outliers.

5.1 Mathematical Formulas

- Standardization: $x' = (x - \mu) / \sigma$
- Min-Max Normalization: $x' = (x - x_min) / (x_max - x_min)$
- Robust Scaling: $x' = (x - median(x)) / IQR(x)$

5.2 Working Principle

The pre-processing pipeline is applied column-wise: numeric columns pass through an imputer followed by a scaler, while categorical columns pass through an imputer (mode) followed by an encoder. scikit-learn's ColumnTransformer / Pipeline objects allow these transformations to be composed and applied consistently to both training and test data, preventing data leakage.

5.3 Types / Classification

Category	Types
Missing value handling	Deletion, Mean/Median/Mode Imputation, KNN Imputation, Iterative Imputation
Encoding	Label Encoding, One-Hot Encoding, Ordinal Encoding, Target Encoding
Scaling	Standardization (Z-score), Min-Max Normalization, Robust Scaling, MaxAbs Scaling

5.4 Comparison Table

Technique	Best Suited For	Sensitive to Outliers?	Output Range
Standardization	Algorithms assuming Gaussian data (Logistic Regression, SVM)	Moderately	Mean=0, Std=1
Min-Max Normalization	Neural Networks, distance-based algorithms	Yes (highly)	[0, 1]
Robust Scaling	Data with many outliers	No	Median-centered
One-Hot Encoding	Nominal categorical data, low cardinality	N/A	Binary (0/1) columns
Label Encoding	Ordinal categorical data	N/A	Integer codes

Table 1.1: Comparison of techniques.

5.5 Observation Table

S.No.	Column Name	Data Type	Missing Values (Before)	Technique Applied	Missing Values (After)
1	Column A	Numeric		Mean Imputation	0
2	Column B	Categorical		Mode Imputation + One-Hot	0
3	...	...			

Table 1.2: Observation table (to be filled during the practical session).

5.6 Figures / Diagrams

Figure 1.1: A bar chart showing count of missing values per column before treatment (plotted using seaborn heatmap `sns.heatmap(df.isnull())` or `msno.matrix(df)` from the missingno library) should be inserted here after running the code cell.

Reference link(s): https://scikit-learn.org/stable/modules/preprocessing.html (see "Standardization, or mean removal and variance scaling" diagram) and https://scikit-learn.org/stable/modules/compose.html#column-transformer for the ColumnTransformer pipeline diagram.

6. Algorithm

1. Start.
2. Import required libraries (pandas, numpy, sklearn).
3. Load the open-access dataset (e.g., Titanic dataset from Kaggle / seaborn) into a pandas DataFrame.
4. Explore the dataset: check shape, data types, and count missing values using df.info() and df.isnull().sum().
5. Separate the dataset into numerical and categorical columns.
6. Apply an appropriate imputation strategy (mean/median for numeric, mode for categorical) using SimpleImputer.
7. Encode categorical columns using LabelEncoder or OneHotEncoder.
8. Scale numeric columns using StandardScaler or MinMaxScaler.
9. Combine the processed numeric and categorical columns into a final model-ready DataFrame.
10. Verify the cleaned dataset (no missing values, all numeric, scaled).
11. Save/export the cleaned dataset.
12. Stop.

7. Flowchart

Draw a flowchart representing the algorithm above (Start → Load Data → Pre-process → Fit Model → Predict → Evaluate → Visualize → Stop). A ready-reference flowchart / diagram for this technique can be found at the link below:

Reference link(s): https://scikit-learn.org/stable/modules/preprocessing.html (see "Standardization, or mean removal and variance scaling" diagram) and https://scikit-learn.org/stable/modules/compose.html#column-transformer for the ColumnTransformer pipeline diagram.

8. Program / Source Code (Python – Jupyter Notebook / Google Colab)

```
# Experiment 1: Feature Engineering - Missing Values, Encoding, Scaling
import pandas as pd
import numpy as np
import seaborn as sns
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# 1. Load open-access dataset (Titanic dataset via seaborn)
df = sns.load_dataset('titanic')
print("Shape:", df.shape)
print(df.isnull().sum())

# 2. Select relevant columns
df = df[['survived','pclass','sex','age','sibsp','parch','fare','embarked']]

num_cols = ['age', 'fare', 'sibsp', 'parch']
cat_cols = ['pclass', 'sex', 'embarked']

# 3. Build pre-processing pipelines
```

```
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer([
    ('num', num_pipeline, num_cols),
    ('cat', cat_pipeline, cat_cols)
])

# 4. Fit-transform
processed_array = preprocessor.fit_transform(df.drop('survived', axis=1))
feature_names = num_cols + list(
    preprocessor.named_transformers_['cat']['encoder'].get_feature_names_out(cat_cols)
)
processed_df = pd.DataFrame(processed_array, columns=feature_names)
processed_df['survived'] = df['survived'].values

print("\nMissing values after processing:\n", processed_df.isnull().sum().sum())
print(processed_df.head())
```

9. Sample Input

Titanic dataset (891 rows, 8 selected columns) loaded via seaborn.load_dataset('titanic'), containing missing values in 'age' (~177) and 'embarked' (2).

10. Expected Output

A processed DataFrame with 0 missing values, numeric columns standardized (mean≈0, std≈1), and categorical columns one-hot encoded, ready to be used as model input.

11. Result

A clean, model-ready dataset was successfully generated after handling missing values, encoding categorical variables, and scaling numerical attributes.

12. Applications

- Pre-processing tabular data for credit risk / fraud-detection models.
- Preparing clinical datasets for disease diagnosis models.
- Standardising survey/marketing data before clustering or classification.

13. Advantages

- Improves model accuracy, stability and convergence speed.
- Prevents features with larger numeric ranges from dominating the learning process.
- Enables the use of categorical information in numeric ML algorithms.

14. Disadvantages

- Improper imputation may introduce bias.
- One-hot encoding increases dimensionality for high-cardinality features.
- Scaling parameters (mean, std, min, max) must be learned only on training data to avoid data leakage.

15. Viva Voce Questions

1. What is feature engineering and why is it necessary?
2. What are the different types of missing data (MCAR, MAR, MNAR)?
3. Differentiate between mean imputation and median imputation. When would you prefer each?
4. What is the difference between Label Encoding and One-Hot Encoding?
5. Why does One-Hot Encoding increase the dimensionality of a dataset?
6. What is the difference between Standardization and Normalization?
7. Why should scalers be fit only on training data and not on the full dataset?
8. What is data leakage, and how does improper pre-processing cause it?
9. Which ML algorithms are sensitive to feature scaling, and which are not?
10. What is the curse of dimensionality?
11. What is the purpose of a ColumnTransformer in scikit-learn?
12. How would you handle a categorical column with 1000 unique values?
13. What is Robust Scaling and when should it be used instead of Standardization?
14. What is an Ordinal Encoder and how does it differ from a Label Encoder?

16. Conclusion

Basic feature engineering techniques — missing value imputation, categorical encoding, and numerical scaling — were successfully applied to an open-access dataset, resulting in a clean and consistent dataset suitable for downstream machine learning modelling.

Experiment No. 2

Dimensionality Reduction using PCA and LDA

---

1. Aim / Problem Statement

To make use of the dimensionality reduction process using Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA) on a suitable open-access high-dimensional dataset and to observe the influence of feature reduction on model performance and computational efficiency.

2. Objectives

- To understand the concept and need for dimensionality reduction.
- To implement PCA as an unsupervised, variance-based dimensionality reduction technique.
- To implement LDA as a supervised, class-separability based dimensionality reduction technique.
- To compare model accuracy and training time before and after dimensionality reduction.

3. Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries: numpy, pandas, scikit-learn (decomposition.PCA, discriminant_analysis.LDA), matplotlib

4. Introduction

4.1 Concept

Dimensionality reduction is the process of reducing the number of input features (dimensions) in a dataset while retaining as much meaningful information as possible. High-dimensional datasets (many columns/features) suffer from the "curse of dimensionality", increased computational cost, multicollinearity, and higher risk of overfitting. Dimensionality reduction techniques project the original high-dimensional data onto a lower-dimensional subspace.

4.2 Background

PCA was introduced by Karl Pearson (1901) and later formalised by Harold Hotelling (1933) as a method to find orthogonal directions of maximum variance. LDA was originally developed by Ronald Fisher (1936) for two-class problems (Fisher's Linear Discriminant) and later generalised to multi-class problems. Both remain foundational techniques in statistics and machine learning for feature reduction, visualization, and noise removal.

4.3 Applications

- Face recognition (Eigenfaces use PCA).
- Gene-expression / genomics data analysis with thousands of features.
- Image compression and denoising.
- Visualizing high-dimensional data in 2D/3D scatter plots.
- Pre-processing before classification to reduce overfitting and training time.

4.4 Advantages

- Reduces computational cost and storage requirements.
- Removes multicollinearity among correlated features.
- Helps visualize high-dimensional data.
- Can improve model generalisation by removing noisy, low-variance dimensions.

4.5 Limitations

- PCA is unsupervised and does not consider class labels, so it may discard discriminative information.
- LDA assumes classes are normally distributed with equal covariance, which may not hold in practice.
- Reduced components are linear combinations of original features and lose direct interpretability.
- LDA can produce at most (C−1) components, where C is the number of classes.

5. Theory

Principal Component Analysis (PCA): PCA is an unsupervised linear transformation technique that identifies the directions (principal components) along which the variance of the data is maximised. It works by computing the covariance matrix of the standardized data, finding its eigenvectors and eigenvalues, and projecting the data onto the eigenvectors corresponding to the largest eigenvalues.

Linear Discriminant Analysis (LDA): LDA is a supervised linear transformation technique that seeks the projection which maximises the ratio of between-class variance to within-class variance, thereby maximising class separability. Unlike PCA, LDA uses class label information during the transformation.

Both techniques transform the original d-dimensional feature space into a new k-dimensional space ($k < d$), where the new axes (components/discriminants) are linear combinations of the original features.

5.1 Mathematical Formulas

- PCA Covariance Matrix: $\Sigma = (1/n) \Sigma (x_i - \bar{x})(x_i - \bar{x})^T$
- PCA Eigen decomposition: $\Sigma v = \lambda v$ (v = eigenvector/principal component, λ = eigenvalue/variance explained)
- Explained Variance Ratio: $\lambda_i / \Sigma\lambda$□
- LDA Between-class scatter: $S_B = \Sigma N_i(\mu_i - \mu)(\mu_i - \mu)^T$

- LDA Within-class scatter: $S_W = \Sigma \Sigma (x - \mu_i)(x - \mu_i)^T$
- LDA Objective: $J(w) = (w^T S_B w) / (w^T S_W w)$ (maximised over projection vector w)

5.2 Working Principle

PCA: (1) Standardize data, (2) compute covariance matrix, (3) compute eigenvalues/eigenvectors, (4) sort eigenvectors by decreasing eigenvalue, (5) select top-k eigenvectors, (6) project data. LDA: (1) Compute mean vectors of each class, (2) compute within-class and between-class scatter matrices, (3) compute eigenvectors of $S_W^{-1} S_B$, (4) select top (C−1) eigenvectors, (5) project data.

5.3 Types / Classification

Category	Types
PCA variants	Standard PCA, Kernel PCA (non-linear), Incremental PCA, Sparse PCA
LDA variants	Standard LDA (Fisher's LDA), Quadratic Discriminant Analysis (QDA), Regularized LDA

5.4 Comparison Table

Aspect	PCA	LDA
Type	Unsupervised	Supervised
Objective	Maximise variance	Maximise class separability
Uses class labels?	No	Yes
Max components	min(n_samples, n_features)	C − 1 (C = number of classes)
Best used for	Noise reduction, visualization, compression	Classification pre-processing

Table 2.1: Comparison of techniques.

5.5 Observation Table

S.No.	Technique	No. of Components	Explained Variance / Accuracy	Training Time (sec)
1	Original (all features)	-		
2	PCA	2/3		
3	LDA	C-1		

Table 2.2: Observation table (to be filled during the practical session).

5.6 Figures / Diagrams

Figure 2.1: Scree plot of explained variance ratio vs. number of principal components (plt.plot(np.cumsum(pca.explained_variance_ratio_))). Figure 2.2: 2-D scatter plot of the

first two principal components / discriminants, coloured by class label, should be inserted here after execution.

Reference link(s):
https://scikit-learn.org/stable/auto_examples/decomposition/plot_pca_iris.html and https://scikit-learn.org/stable/auto_examples/decomposition/plot_pca_vs_lda.html for reference PCA vs LDA diagrams on the Iris dataset.

6. Algorithm

1. Start.
2. Import libraries and load a high-dimensional open-access dataset (e.g., Wine or Digits dataset from sklearn.datasets).
3. Standardize the features using StandardScaler.
4. Split data into training and testing sets.
5. Train a baseline classifier (e.g., Logistic Regression) on the original full-dimensional data; record accuracy and training time.
6. Apply PCA to reduce dimensions to k components; train the same classifier on reduced data; record accuracy and training time.
7. Apply LDA to reduce dimensions to (C−1) components; train the same classifier; record accuracy and training time.
8. Compare accuracy, explained variance, and training time across the three cases.
9. Plot the explained variance ratio (scree plot) and 2-D projections.
10. Stop.

7. Flowchart

Draw a flowchart representing the algorithm above (Start → Load Data → Pre-process → Fit Model → Predict → Evaluate → Visualize → Stop). A ready-reference flowchart / diagram for this technique can be found at the link below:

Reference link(s):
https://scikit-learn.org/stable/auto_examples/decomposition/plot_pca_iris.html and https://scikit-learn.org/stable/auto_examples/decomposition/plot_pca_vs_lda.html for reference PCA vs LDA diagrams on the Iris dataset.

8. Program / Source Code (Python – Jupyter Notebook / Google Colab)

```
# Experiment 2: Dimensionality Reduction using PCA and LDA

import time

import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import load_wine
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 1. Load high-dimensional open-access dataset
data = load_wine()
X, y = data.data, data.target
print("Original shape:", X.shape)

# 2. Standardize
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

def evaluate(Xtr, Xte, label):
    t0 = time.time()
    clf = LogisticRegression(max_iter=1000)
    clf.fit(Xtr, y_train)
    t1 = time.time()
    acc = accuracy_score(y_test, clf.predict(Xte))
    print(f"{label}: Accuracy={acc:.4f}, Training time={t1-t0:.4f}s,
Dimensions={Xtr.shape[1]}")
    return acc, t1 - t0
```

```
# Baseline - full dimensions
evaluate(X_train_s, X_test_s, "Original (13 features)")


# PCA
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_s)
X_test_pca = pca.transform(X_test_s)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
evaluate(X_train_pca, X_test_pca, "PCA (2 components)")


# LDA
lda = LDA(n_components=2)
X_train_lda = lda.fit_transform(X_train_s, y_train)
X_test_lda = lda.transform(X_test_s)
evaluate(X_train_lda, X_test_lda, "LDA (2 components)")


# Scree plot
pca_full = PCA().fit(X_train_s)
plt.plot(np.cumsum(pca_full.explained_variance_ratio_), marker='o')
plt.xlabel('Number of Components'); plt.ylabel('Cumulative Explained Variance')
plt.title('PCA Scree Plot'); plt.grid(True); plt.show()
```

9. Sample Input

Wine dataset from sklearn.datasets (178 samples, 13 numeric features, 3 classes).

10. Expected Output

Console output comparing accuracy and training time for Original, PCA (2 components) and LDA (2 components); a scree plot showing cumulative explained variance.

11. Result

Dimensionality reduction using PCA and LDA was successfully implemented; classification accuracy and computational efficiency were compared across the original and reduced feature spaces.

12. Applications

- Face recognition and image compression.
- Gene expression / bioinformatics analysis.
- Data visualization of high-dimensional datasets.
- Noise reduction before clustering or classification.

13. Advantages

- Reduces overfitting and computational cost.
- Removes redundant/correlated features.
- Improves visualization of complex datasets.

14. Disadvantages

- Loss of interpretability of original features.
- PCA may discard components useful for classification since it ignores labels.
- LDA is limited to at most C−1 components and assumes Gaussian class distributions.

15. Viva Voce Questions

1. What is dimensionality reduction and why is it needed?
2. Differentiate between PCA and LDA.
3. What is an eigenvalue and eigenvector in the context of PCA?
4. What does the explained variance ratio represent?
5. Why is LDA called a supervised technique while PCA is unsupervised?
6. What is the maximum number of components possible in LDA for a C-class problem?
7. What is the curse of dimensionality?
8. Why is standardization important before applying PCA?
9. What is Kernel PCA and when is it used?
10. How do you decide the optimal number of principal components?
11. What are between-class scatter and within-class scatter matrices in LDA?
12. Can PCA be used for feature selection? Why or why not?
13. What are the assumptions made by LDA?
14. How does dimensionality reduction affect model training time?

16. Conclusion

PCA and LDA were successfully applied on a high-dimensional open-access dataset, demonstrating that dimensionality reduction can substantially reduce computational time while maintaining (and in the case of LDA, sometimes improving) classification accuracy.

Experiment No. 3

Multiple Linear Regression and Model Evaluation using MSE, RMSE and R^2

---

1. Aim / Problem Statement

To develop a multiple linear regression model using a suitable real-world open-access dataset and to assess its predictive capability using regression evaluation metrics such as MSE, RMSE, and R^2.

2. Objectives

- To understand the concept of multiple linear regression.
- To build a regression model using multiple independent variables to predict a continuous dependent variable.
- To evaluate model performance using MSE, RMSE, and R^2 metrics.
- To interpret regression coefficients and residuals.

3. Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries: pandas, numpy, scikit-learn (linear_model.LinearRegression, metrics), matplotlib, seaborn

4. Introduction

4.1 Concept

Multiple Linear Regression (MLR) is a supervised learning technique used to model the linear relationship between a dependent (target) variable and two or more independent (predictor) variables. It extends simple linear regression, which uses a single predictor, to multiple predictors, allowing more realistic modelling of real-world phenomena where an outcome is influenced by several factors simultaneously.

4.2 Background

Regression analysis dates back to Sir Francis Galton (1886), who coined the term "regression" while studying heredity. Multiple regression was later formalised through the work of Karl Pearson and R.A. Fisher in the early 20th century, and today it forms a foundational algorithm in statistics, econometrics, and machine learning.

4.3 Applications

- Predicting house prices from features such as area, location, and number of rooms.
- Predicting sales revenue from advertising spend across multiple channels.
- Predicting student performance based on study hours, attendance, and prior grades.
- Medical cost prediction based on age, BMI, smoking status, etc.

4.4 Advantages

- Simple, interpretable, and computationally efficient.
- Coefficients directly indicate the influence/direction of each predictor.
- Works well when the relationship between variables is approximately linear.

4.5 Limitations

- Assumes a linear relationship between predictors and target.
- Sensitive to multicollinearity among independent variables.
- Sensitive to outliers.
- Cannot capture complex non-linear relationships without transformation.

5. Theory

Multiple Linear Regression models the target variable y as a linear combination of n independent variables x_1, x_2, ..., x_n, plus an intercept and an error term. The model parameters (coefficients) are typically estimated using the Ordinary Least Squares (OLS) method, which minimises the sum of squared residuals between the actual and predicted values.

Key assumptions of MLR: linearity, independence of errors, homoscedasticity (constant variance of errors), normality of residuals, and absence of multicollinearity among predictors.

5.1 Mathematical Formulas

- Model equation: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n + \varepsilon$
- OLS objective: minimise $\Sigma(y_i - \hat{y}_i)^2$
- Mean Squared Error: $MSE = (1/n)\ \Sigma(y_i - \hat{y}_i)^2$
- Root Mean Squared Error: $RMSE = \sqrt{MSE}$
- R-squared: $R^2 = 1 - (SS_res / SS_tot) = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2$
- Adjusted R^2: $R^2_adj = 1 - [(1 - R^2)(n - 1) / (n - p - 1)]$

5.2 Working Principle

The algorithm estimates coefficients β that minimise the residual sum of squares between predicted and actual target values, typically solved in closed form using the Normal Equation $\beta = (X^T X)^{-1} X^T y$, or iteratively via Gradient Descent for large datasets. Once fitted, the model predicts new outcomes as a weighted sum of the input features.

5.3 Types / Classification

Category	Types
Estimation methods	Ordinary Least Squares (OLS), Gradient Descent, Normal Equation
Regression family	Simple Linear Regression, Multiple Linear Regression, Polynomial Regression

5.4 Comparison Table

Metric	Formula (short)	Interpretation	Ideal Value
MSE	Avg. squared error	Penalises large errors heavily	Close to 0
RMSE	√MSE	Error in same units as target	Close to 0
R^2	1 − SS_res/SS_tot	Proportion of variance explained	Close to 1
Adjusted R^2	Penalised R^2 for no. of predictors	Corrects for overfitting with many features	Close to 1

Table 3.1: Comparison of techniques.

5.5 Observation Table

S.No.	Feature Set	MSE	RMSE	R^2
1	All features			
2	Reduced feature subset			

Table 3.2: Observation table (to be filled during the practical session).

5.6 Figures / Diagrams

Figure 3.1: Scatter plot of Actual vs Predicted values (should form a line close to y=x).
Figure 3.2: Residual plot (residuals vs predicted values) to check homoscedasticity — insert after running the code cell.

Reference link(s):
https://scikit-learn.org/stable/modules/linear_model.html#ordinary-least-squares (reference diagram/description of the OLS fitting process).

6. Algorithm

1. Start.
2. Import required libraries and load a real-world open-access dataset (e.g., California Housing dataset from sklearn.datasets).
3. Explore data: check for missing values, data types, and correlations between features.
4. Separate features (X) and target variable (y).
5. Split the dataset into training and testing sets (e.g., 80:20).
6. Initialize and train a LinearRegression model on the training data.
7. Predict target values on the test set.
8. Compute evaluation metrics: MSE, RMSE, and R^2 between actual and predicted values.
9. Visualize actual vs predicted values and residuals.
10. Interpret model coefficients.

11. Stop.

7. Flowchart

Draw a flowchart representing the algorithm above (Start → Load Data → Pre-process → Fit Model → Predict → Evaluate → Visualize → Stop). A ready-reference flowchart / diagram for this technique can be found at the link below:

Reference link(s):
https://scikit-learn.org/stable/modules/linear_model.html#ordinary-least-squares (reference diagram/description of the OLS fitting process).

8. Program / Source Code (Python – Jupyter Notebook / Google Colab)

```
# Experiment 3: Multiple Linear Regression and Evaluation (MSE, RMSE, R2)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 1. Load real-world open-access dataset
housing = fetch_california_housing(as_frame=True)
df = housing.frame
X = df.drop('MedHouseVal', axis=1)
y = df['MedHouseVal']

# 2. Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Train multiple linear regression model
model = LinearRegression()
```

```
model.fit(X_train, y_train)


# 4. Predict
y_pred = model.predict(X_test)


# 5. Evaluate
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)


print("Coefficients:", dict(zip(X.columns, model.coef_)))
print("Intercept:", model.intercept_)
print(f"MSE  = {mse:.4f}")
print(f"RMSE = {rmse:.4f}")
print(f"R2   = {r2:.4f}")


# 6. Visualization
plt.scatter(y_test, y_pred, alpha=0.4)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--')
plt.xlabel('Actual'); plt.ylabel('Predicted'); plt.title('Actual vs Predicted')
plt.show()
```

9. Sample Input

California Housing dataset (20,640 samples, 8 numeric features such as MedInc, HouseAge, AveRooms; target = MedHouseVal).

10. Expected Output

Fitted regression coefficients, intercept, and metric values (e.g., MSE≈0.53, RMSE≈0.73, R^2≈0.60 approximately, values vary by split); scatter plot of actual vs predicted values.

11. Result

A multiple linear regression model was successfully built and its predictive performance evaluated using MSE, RMSE, and R^2 metrics.

12. Applications

- Real-estate price prediction.
- Sales and demand forecasting.
- Risk-scoring in insurance and finance.
- Academic performance prediction.

13. Advantages

- Easy to implement and interpret.
- Computationally inexpensive.
- Provides insight into feature importance via coefficients.

14. Disadvantages

- Assumes linearity; performs poorly on non-linear relationships.
- Sensitive to multicollinearity and outliers.
- R^2 can be misleadingly high with many predictors (mitigated by Adjusted R^2).

15. Viva Voce Questions

1. What is multiple linear regression?
2. State the assumptions of linear regression.
3. What is the difference between MSE and RMSE?
4. What does an R^2 value of 0.85 indicate?
5. What is the difference between R^2 and Adjusted R^2?
6. What is multicollinearity and how can it be detected?
7. What is the Ordinary Least Squares method?
8. Why is RMSE preferred over MSE for interpretation?
9. What is a residual, and what should a residual plot ideally look like?
10. How does an outlier affect a linear regression model?
11. What is the difference between simple and multiple linear regression?
12. How do you interpret a regression coefficient?
13. What is heteroscedasticity?
14. Can R^2 be negative? What does that mean?

16. Conclusion

A multiple linear regression model was developed on a real-world open-access dataset, and its predictive capability was quantitatively assessed using MSE, RMSE, and R^2, demonstrating the relationship between multiple predictors and the target variable.

Experiment No. 4

Multiple Linear Regression: Predictive Capability Assessment on an Alternate Dataset

---

1. Aim / Problem Statement

To develop a multiple linear regression model using a suitable real-world open-access dataset and to assess its predictive capability using regression evaluation metrics such as MSE, RMSE, and R^2.

2. Objectives

- To reinforce the concept of multiple linear regression on a different real-world dataset.
- To perform exploratory data analysis (EDA) and correlation analysis prior to model building.
- To build and train a multiple linear regression model.
- To evaluate and interpret the model using MSE, RMSE, and R^2, and compare with Experiment 3.

3. Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries: pandas, numpy, scikit-learn, matplotlib, seaborn

4. Introduction

4.1 Concept

This experiment reinforces multiple linear regression concepts by applying the technique to an alternate real-world open-access dataset (the Diabetes dataset), allowing comparison of model behaviour, coefficient interpretation, and metric values across different data domains and feature correlations.

4.2 Background

The Diabetes dataset used here was collected by Efron, Hastie, Johnstone and Tibshirani (2004) and is a standard benchmark dataset bundled with scikit-learn for regression tasks, containing ten baseline physiological variables measured on diabetes patients.

4.3 Applications

- Disease progression prediction in healthcare analytics.
- Predicting clinical/biological outcomes from multiple physiological measurements.
- General-purpose benchmarking of regression algorithms.

4.4 Advantages

- Provides a second, independent validation of the MLR technique.

- Highlights how feature correlation structure affects coefficient stability.
- Reinforces evaluation-metric interpretation across domains.

4.5 Limitations

- Same linearity, multicollinearity, and outlier-sensitivity limitations as Experiment 3.
- Small sample size (442 rows) can lead to higher variance in metric estimates.

5. Theory

As in Experiment 3, the model fits $y = \beta_0 + \beta_1 x_1 + ... + \beta_n x_n + \varepsilon$ using Ordinary Least Squares. In this experiment, particular emphasis is placed on correlation analysis (using a heatmap) prior to model fitting, to identify potential multicollinearity among the ten physiological predictors (age, sex, BMI, blood pressure, and six blood serum measurements), and on comparing evaluation metrics against those obtained in Experiment 3.

5.1 Mathematical Formulas

- Model equation: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n + \varepsilon$
- MSE = $(1/n)\ \Sigma(y_i - \hat{y}_i)^2$
- RMSE = $\sqrt{MSE}$
- $R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2$
- Variance Inflation Factor (multicollinearity check): $VIF_i = 1 / (1 - R_i^2)$

5.2 Working Principle

Correlation heatmap is generated first to inspect inter-feature relationships; the OLS-based LinearRegression estimator is then fit on the training split, coefficients are examined for sign/magnitude consistency with domain knowledge, and predictions on the held-out test set are evaluated with MSE, RMSE and R^2.

5.3 Types / Classification

Category	Types
Correlation analysis tools	Pearson correlation heatmap, VIF
Estimation methods	OLS (closed form), Gradient Descent

5.4 Comparison Table

Experiment	Dataset	No. of Features	Typical R^2 Range
Experiment 3	California Housing	8	0.55 - 0.65
Experiment 4	Diabetes	10	0.40 - 0.55

Table 4.1: Comparison of techniques.

5.5 Observation Table

S.No.	Model	MSE	RMSE	R^2
1	MLR - all 10 features			

2	MLR - top correlated features only			

Table 4.2: Observation table (to be filled during the practical session).

5.6 Figures / Diagrams

Figure 4.1: Correlation heatmap of all predictor variables (sns.heatmap(df.corr(), annot=True)). Figure 4.2: Actual vs Predicted scatter plot for the Diabetes dataset — insert after execution.

Reference link(s): https://scikit-learn.org/stable/auto_examples/linear_model/plot_ols.html (reference OLS fitting example and diagram).

6. Algorithm

1. Start.
2. Import libraries and load the Diabetes dataset using sklearn.datasets.load_diabetes().
3. Convert to a pandas DataFrame; perform EDA and plot a correlation heatmap.
4. Separate features (X) and target (y).
5. Split data into training and testing sets.
6. Train a LinearRegression model on the training set.
7. Predict on the test set.
8. Compute MSE, RMSE, and R^2.
9. Compare results with Experiment 3 and interpret coefficients.
10. Stop.

7. Flowchart

Draw a flowchart representing the algorithm above (Start → Load Data → Pre-process → Fit Model → Predict → Evaluate → Visualize → Stop). A ready-reference flowchart / diagram for this technique can be found at the link below:

Reference link(s): https://scikit-learn.org/stable/auto_examples/linear_model/plot_ols.html (reference OLS fitting example and diagram).

8. Program / Source Code (Python – Jupyter Notebook / Google Colab)

```
# Experiment 4: Multiple Linear Regression on the Diabetes dataset

import numpy as np

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 1. Load dataset
data = load_diabetes(as_frame=True)
df = data.frame
X = df.drop('target', axis=1)
y = df['target']

# 2. Correlation heatmap
plt.figure(figsize=(9,7))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap - Diabetes Dataset')
plt.show()

# 3. Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Fit model
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# 5. Evaluate
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
```

```
r2 = r2_score(y_test, y_pred)
print(f"MSE={mse:.4f}, RMSE={rmse:.4f}, R2={r2:.4f}")
print("Coefficients:", dict(zip(X.columns, model.coef_)))
```

9. Sample Input

Diabetes dataset from sklearn.datasets (442 samples, 10 baseline physiological features; target = disease progression measure after one year).

10. Expected Output

Correlation heatmap; regression coefficients; evaluation metrics (typical R^2 around 0.45-0.55 depending on split).

11. Result

A multiple linear regression model was successfully developed and evaluated on the Diabetes dataset, and its performance was compared with Experiment 3.

12. Applications

- Disease-progression prediction.
- Healthcare risk assessment.
- Comparative benchmarking across datasets/domains.

13. Advantages

- Reinforces MLR concepts on a second dataset.
- Correlation heatmap aids early detection of multicollinearity.
- Simple and interpretable.

14. Disadvantages

- Small dataset size increases metric variance.
- Same linear-assumption limitations as Experiment 3.

15. Viva Voce Questions

1. Why was correlation analysis performed before model building?
2. What does a high correlation between two predictors indicate?
3. What is the Variance Inflation Factor (VIF) and how is it used?
4. How does dataset size affect the reliability of R^2?
5. Why might R^2 differ significantly between two datasets using the same algorithm?
6. What steps would you take if two predictors are highly correlated?
7. What is the difference between correlation and causation?
8. How is the Diabetes dataset target variable defined?

9. What does a negative regression coefficient indicate?
10. How can regularization help when multicollinearity is present?
11. What is the effect of removing a highly correlated feature on model coefficients?
12. Compare the R^2 obtained in Experiment 3 and Experiment 4 and justify the difference.

16. Conclusion

The multiple linear regression methodology was validated on a second real-world dataset (Diabetes), reinforcing the workflow of model building and evaluation using MSE, RMSE and R^2, while correlation analysis highlighted the importance of checking multicollinearity.

Experiment No. 5

Regularization Techniques: Ridge and Lasso Regression

---

1. Aim / Problem Statement

To classify/predict data using regularization techniques such as Ridge and Lasso regression on a suitable multivariate open-access dataset and to observe the influence of regularization on model overfitting and multicollinearity.

2. Objectives

- To understand the concept of overfitting and regularization in linear models.
- To implement Ridge Regression (L2 regularization) and Lasso Regression (L1 regularization).
- To observe the effect of the regularization parameter (alpha/lambda) on model coefficients.
- To compare Ridge, Lasso and plain Linear Regression in the presence of multicollinearity.

3. Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries: pandas, numpy, scikit-learn (linear_model.Ridge, Lasso), matplotlib

4. Introduction

4.1 Concept

Regularization is a technique used to prevent overfitting in regression models by adding a penalty term to the loss function that discourages excessively large coefficient values. Ridge Regression adds an L2 penalty (sum of squared coefficients), while Lasso Regression adds an L1 penalty (sum of absolute coefficients). Both shrink coefficients toward zero, with Lasso capable of shrinking some coefficients exactly to zero, thereby performing automatic feature selection.

4.2 Background

Ridge Regression was introduced by Hoerl and Kennard (1970) to address multicollinearity in Ordinary Least Squares regression. Lasso (Least Absolute Shrinkage and Selection Operator) was proposed by Robert Tibshirani (1996) as an alternative that simultaneously performs regularization and variable selection. Elastic Net, combining both penalties, was later introduced by Zou and Hastie (2005).

4.3 Applications

- Genomics: selecting a small number of informative genes from thousands of predictors (Lasso).

- Finance: predicting stock returns from many correlated economic indicators (Ridge).
- Any high-dimensional regression problem prone to overfitting or multicollinearity.

4.4 Advantages

- Reduces overfitting, improving generalisation to unseen data.
- Ridge handles multicollinearity effectively by shrinking correlated coefficients together.
- Lasso performs automatic feature selection by zeroing out irrelevant features.

4.5 Limitations

- Requires tuning of the regularization strength (alpha) via cross-validation.
- Ridge does not perform feature selection (coefficients shrink but rarely become exactly zero).
- Lasso can arbitrarily select one feature among a group of highly correlated features.

5. Theory

Both Ridge and Lasso modify the Ordinary Least Squares cost function by adding a penalty term controlled by a hyperparameter α (also called λ):

Ridge minimises: $\Sigma(y_i - \hat{y}_i)^2 + \alpha \cdot \Sigma\beta\square^2$ (L2 penalty)

Lasso minimises: $\Sigma(y_i - \hat{y}_i)^2 + \alpha \cdot \Sigma|\beta\square|$ (L1 penalty)

As α increases, the penalty dominates, shrinking coefficients toward zero — reducing model variance at the cost of a small increase in bias (bias-variance trade-off). At α = 0, both reduce to Ordinary Least Squares regression.

5.1 Mathematical Formulas

- OLS Loss: $L = \Sigma(y_i - \hat{y}_i)^2$
- Ridge Loss (L2): $L = \Sigma(y_i - \hat{y}_i)^2 + \alpha\ \Sigma\beta\square^2$
- Lasso Loss (L1): $L = \Sigma(y_i - \hat{y}_i)^2 + \alpha\ \Sigma|\beta\square|$
- Elastic Net Loss: $L = \Sigma(y_i - \hat{y}_i)^2 + \alpha_1\Sigma|\beta\square| + \alpha_2\Sigma\beta\square^2$

5.2 Working Principle

The regularized loss function is minimised using coordinate descent (Lasso) or a closed-form/gradient-based solution (Ridge). The penalty term constrains the coefficient magnitudes, geometrically corresponding to constraining the solution to lie within an L1-norm diamond (Lasso) or L2-norm sphere (Ridge) around the origin, which is why Lasso's diamond-shaped constraint region tends to intersect the loss contours at the axes (producing exact zero coefficients).

5.3 Types / Classification

Category	Types
Regularization types	L1 (Lasso), L2 (Ridge), L1+L2 (Elastic Net)

5.4 Comparison Table

Aspect	Ridge (L2)	Lasso (L1)
Penalty	Sum of squared coefficients	Sum of absolute coefficients
Feature selection	No (shrinks but rarely zero)	Yes (can zero out coefficients)
Handles multicollinearity	Very well	Reasonably well
Best when	Many small/medium predictors all relevant	Few predictors truly relevant (sparse)

Table 5.1: Comparison of techniques.

5.5 Observation Table

S.No.	Alpha (α)	Ridge R^2	Ridge - No. of non-zero coeffs	Lasso R^2	Lasso - No. of non-zero coeffs
1	0.01				
2	1				
3	10				
4	100				

Table 5.2: Observation table (to be filled during the practical session).

5.6 Figures / Diagrams

Figure 5.1: Plot of coefficient magnitude vs. alpha for Ridge and Lasso (coefficient path / regularization path plot). Figure 5.2: Bar chart comparing number of non-zero coefficients for Ridge vs Lasso at increasing alpha — insert after execution.

Reference link(s):
https://scikit-learn.org/stable/auto_examples/linear_model/plot_ridge_path.html and https://scikit-learn.org/stable/auto_examples/linear_model/plot_lasso_coordinate_descent_path.html for reference regularization path diagrams.

6. Algorithm

1. Start.
2. Import libraries and load a multivariate open-access dataset (e.g., California Housing or a synthetic multicollinear dataset).
3. Standardize the feature set (mandatory for fair regularization).
4. Split data into training and testing sets.
5. Fit a plain Linear Regression model as baseline; note coefficients and R^2.
6. Fit Ridge Regression models for a range of alpha values; record coefficients and R^2 for each.
7. Fit Lasso Regression models for the same range of alpha values; record coefficients (noting zeros) and R^2.
8. Plot the regularization path (coefficient magnitude vs alpha) for both Ridge and Lasso.

9. Use cross-validation (RidgeCV/LassoCV) to select the optimal alpha.
10. Compare baseline, Ridge, and Lasso performance and overfitting behaviour.
11. Stop.

7. Flowchart

Draw a flowchart representing the algorithm above (Start → Load Data → Pre-process → Fit Model → Predict → Evaluate → Visualize → Stop). A ready-reference flowchart / diagram for this technique can be found at the link below:

Reference link(s):
https://scikit-learn.org/stable/auto_examples/linear_model/plot_ridge_path.html and https://scikit-learn.org/stable/auto_examples/linear_model/plot_lasso_coordinate_descent_path.html for reference regularization path diagrams.

8. Program / Source Code (Python – Jupyter Notebook / Google Colab)

```
# Experiment 5: Ridge and Lasso Regression

import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression, Ridge, Lasso, RidgeCV, LassoCV

from sklearn.metrics import r2_score, mean_squared_error


# 1. Load data

data = fetch_california_housing(as_frame=True)

X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


scaler = StandardScaler()

X_train_s = scaler.fit_transform(X_train)

X_test_s = scaler.transform(X_test)
```

```
# 2. Baseline OLS
lr = LinearRegression().fit(X_train_s, y_train)
print("OLS R2:", r2_score(y_test, lr.predict(X_test_s)))

# 3. Ridge and Lasso across alphas
alphas = [0.01, 0.1, 1, 10, 100]
ridge_coefs, lasso_coefs = [], []
for a in alphas:
    ridge = Ridge(alpha=a).fit(X_train_s, y_train)
    lasso = Lasso(alpha=a, max_iter=10000).fit(X_train_s, y_train)
    ridge_coefs.append(ridge.coef_)
    lasso_coefs.append(lasso.coef_)
    print(f"alpha={a:>6}: Ridge R2={r2_score(y_test, ridge.predict(X_test_s)):.4f} "
          f"| Lasso R2={r2_score(y_test, lasso.predict(X_test_s)):.4f} "
          f"| Lasso non-zero coeffs={np.sum(lasso.coef_ != 0)}")

# 4. Optimal alpha via cross-validation
ridge_cv = RidgeCV(alphas=alphas).fit(X_train_s, y_train)
lasso_cv = LassoCV(alphas=alphas, max_iter=10000).fit(X_train_s, y_train)
print("Best Ridge alpha:", ridge_cv.alpha_)
print("Best Lasso alpha:", lasso_cv.alpha_)

# 5. Regularization path plot
ridge_coefs = np.array(ridge_coefs)
plt.plot(alphas, ridge_coefs)
plt.xscale('log'); plt.xlabel('alpha'); plt.ylabel('Coefficient value')
plt.title('Ridge Regularization Path'); plt.show()
```

9. Sample Input

California Housing dataset (20,640 samples, 8 numeric features), standardized before use.

10. Expected Output

R^2 scores for OLS, Ridge, and Lasso across multiple alpha values; count of non-zero Lasso coefficients decreasing as alpha increases; regularization path plot.

11. Result

Ridge and Lasso regression were successfully implemented, showing that regularization controls overfitting and that Lasso performs automatic feature selection while Ridge retains all features with shrunk coefficients.

12. Applications

- High-dimensional genomics/bioinformatics regression.
- Financial modelling with correlated economic indicators.
- Any regression task with many/correlated features prone to overfitting.

13. Advantages

- Reduces overfitting and improves generalisation.
- Ridge stabilises coefficient estimates under multicollinearity.
- Lasso simplifies models via automatic feature selection.

14. Disadvantages

- Requires careful tuning of alpha (computationally expensive with CV).
- Lasso may inconsistently select among correlated features.
- Both introduce bias in exchange for reduced variance.

15. Viva Voce Questions

1. What is regularization and why is it used?
2. Differentiate between L1 and L2 regularization.
3. Why does Lasso produce sparse (zero) coefficients while Ridge does not?
4. What is the bias-variance trade-off in the context of regularization?
5. What happens to the model when alpha = 0?
6. What happens to the model when alpha is very large?
7. What is Elastic Net regression?
8. How do you choose the optimal value of alpha?
9. What is multicollinearity and how does Ridge address it?
10. Why is feature scaling necessary before applying Ridge/Lasso?
11. What is the geometric interpretation of L1 vs L2 constraint regions?
12. How is RidgeCV/LassoCV different from Ridge/Lasso?
13. Can Lasso be used for feature selection? Explain.

14. What is overfitting, and how do you detect it?

16. Conclusion

Ridge and Lasso regularization techniques were successfully applied to a multivariate open-access dataset, demonstrating their effectiveness in controlling overfitting and addressing multicollinearity compared to ordinary linear regression.

Experiment No. 6

Binary Classification using Logistic Regression: Confusion Matrix, Precision, Recall, and F1-Score

---

1. Aim / Problem Statement

To illustrate the performance of a binary classification model using logistic regression on a suitable open-access dataset through a confusion matrix, precision, recall, and F1-score.

2. Objectives

- To understand logistic regression as a classification algorithm.
- To build a binary classifier using logistic regression on an open-access dataset.
- To evaluate the classifier using a confusion matrix and derived metrics: accuracy, precision, recall, and F1-score.
- To understand the trade-off between precision and recall.

3. Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries: pandas, numpy, scikit-learn (linear_model.LogisticRegression, metrics), matplotlib, seaborn

4. Introduction

4.1 Concept

Logistic Regression is a supervised learning algorithm used for binary (and multi-class) classification. Despite its name, it is a classification algorithm, not a regression algorithm — it models the probability that a given input belongs to a particular class using the logistic (sigmoid) function, which maps any real-valued input to a value between 0 and 1.

4.2 Background

Logistic regression originates from the work of statistician David Cox (1958) and builds upon the logistic function first described by Pierre François Verhulst in the 19th century for modelling population growth. It remains one of the most widely used and interpretable classification algorithms in statistics and machine learning.

4.3 Applications

- Medical diagnosis (disease present/absent).
- Email spam detection.
- Credit default prediction (default/no default).
- Customer churn prediction.

4.4 Advantages

- Simple, fast, and highly interpretable (odds ratios).
- Outputs well-calibrated probabilities.
- Works well when classes are linearly separable and performs well on small-to-medium datasets.

4.5 Limitations

- Assumes a linear decision boundary between classes in log-odds space.
- Sensitive to outliers and multicollinearity.
- Not well suited for complex non-linear class boundaries without feature engineering.

5. Theory

Logistic regression models the probability of the positive class using the sigmoid function applied to a linear combination of the input features. The predicted probability is thresholded (commonly at 0.5) to obtain the final class label. Model parameters are estimated by maximising the likelihood of the observed data (Maximum Likelihood Estimation), typically solved iteratively via gradient descent or Newton's method.

Model evaluation for classification relies on the confusion matrix, a 2×2 table (for binary classification) that cross-tabulates actual vs predicted classes into True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). From this matrix, several performance metrics are derived.

5.1 Mathematical Formulas

- Sigmoid function: $\sigma(z) = 1 / (1 + e^{-z})$
- Logit / log-odds: $z = \beta_0 + \beta_1 x_1 + ... + \beta_\square x_\square$
- Predicted probability: $P(y=1|x) = \sigma(z)$
- Accuracy = (TP + TN) / (TP + TN + FP + FN)
- Precision = TP / (TP + FP)
- Recall (Sensitivity) = TP / (TP + FN)
- F1-score = 2 · (Precision · Recall) / (Precision + Recall)
- Specificity = TN / (TN + FP)

5.2 Working Principle

During training, the logistic regression algorithm iteratively adjusts weights β to maximise the log-likelihood of correctly classifying the training samples. During inference, a new sample's features are combined linearly and passed through the sigmoid function to obtain a probability, which is then compared against a decision threshold (default 0.5) to output the final binary class label.

5.3 Types / Classification

Category	Types
Classification threshold strategies	Default (0.5), ROC-optimal threshold, Precision-Recall optimal threshold

Regularized variants	L1-penalized Logistic Regression, L2-penalized Logistic Regression, Elastic-Net Logistic Regression

5.4 Comparison Table

Metric	Formula	When to Prioritize
Precision	TP/(TP+FP)	When false positives are costly (e.g., spam filter)
Recall	TP/(TP+FN)	When false negatives are costly (e.g., disease detection)
F1-score	Harmonic mean of Precision & Recall	When classes are imbalanced and both errors matter
Accuracy	(TP+TN)/Total	When classes are balanced

Table 6.1: Comparison of techniques.

5.5 Observation Table

Predicted \\ Actual	Positive (1)	Negative (0)
Positive (1)	TP =	FP =
Negative (0)	FN =	TN =

Table 6.2: Observation table (to be filled during the practical session).

5.6 Figures / Diagrams

Figure 6.1: Confusion matrix heatmap (sns.heatmap(confusion_matrix, annot=True)). Figure 6.2: Sigmoid curve illustrating probability output. Figure 6.3: ROC curve with AUC — insert after execution.

Reference link(s):
https://scikit-learn.org/stable/auto_examples/model_selection/plot_confusion_matrix.html and https://scikit-learn.org/stable/modules/model_evaluation.html#confusion-matrix for reference confusion-matrix diagrams.

6. Algorithm

1. Start.
2. Import libraries and load an open-access binary classification dataset (e.g., Breast Cancer Wisconsin dataset from sklearn.datasets).
3. Explore the dataset and check class balance.
4. Split data into features (X) and target (y); split into training and testing sets.
5. Standardize features if required.
6. Train a LogisticRegression model on the training set.
7. Predict class labels and probabilities on the test set.
8. Compute the confusion matrix (TP, TN, FP, FN).

9. Compute accuracy, precision, recall, and F1-score.
10. Plot the confusion matrix heatmap and the ROC curve.
11. Interpret precision-recall trade-offs.
12. Stop.

7. Flowchart

Draw a flowchart representing the algorithm above (Start → Load Data → Pre-process → Fit Model → Predict → Evaluate → Visualize → Stop). A ready-reference flowchart / diagram for this technique can be found at the link below:

Reference link(s):
https://scikit-learn.org/stable/auto_examples/model_selection/plot_confusion_matrix.html and https://scikit-learn.org/stable/modules/model_evaluation.html#confusion-matrix for reference confusion-matrix diagrams.

8. Program / Source Code (Python – Jupyter Notebook / Google Colab)

```
# Experiment 6: Logistic Regression - Binary Classification Evaluation

import numpy as np

import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (confusion_matrix, accuracy_score, precision_score,

                             recall_score, f1_score, classification_report,

                             RocCurveDisplay)


# 1. Load dataset

data = load_breast_cancer()

X, y = data.data, data.target   # 1 = benign, 0 = malignant


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
```

```
                                        random_state=42, stratify=y)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# 2. Train logistic regression
clf = LogisticRegression(max_iter=5000)
clf.fit(X_train_s, y_train)

# 3. Predict
y_pred = clf.predict(X_test_s)
y_prob = clf.predict_proba(X_test_s)[:, 1]

# 4. Evaluate
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1-score :", f1_score(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=data.target_names))

# 5. Visualize
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=data.target_names, yticklabels=data.target_names)
plt.xlabel('Predicted'); plt.ylabel('Actual'); plt.title('Confusion Matrix')
plt.show()
```

```
RocCurveDisplay.from_predictions(y_test, y_prob)
plt.title('ROC Curve'); plt.show()
```

9. Sample Input

Breast Cancer Wisconsin dataset from sklearn.datasets (569 samples, 30 numeric features; binary target: malignant/benign).

10. Expected Output

Confusion matrix, accuracy, precision, recall, F1-score (typically Accuracy ≈ 0.96-0.98 on this dataset); confusion matrix heatmap; ROC curve with AUC close to 1.

11. Result

A binary logistic regression classifier was successfully built and its performance illustrated through a confusion matrix, precision, recall, and F1-score.

12. Applications

- Disease diagnosis and medical screening.
- Fraud/spam detection.
- Credit risk / loan default prediction.
- Customer churn prediction.

13. Advantages

- Interpretable coefficients and probability outputs.
- Computationally efficient for binary classification.
- Confusion-matrix-based metrics give a comprehensive view of classifier performance, especially for imbalanced classes.

14. Disadvantages

- Assumes a linear decision boundary in log-odds space.
- Sensitive to class imbalance if not addressed (e.g., via class_weight).
- Threshold choice (0.5 default) may not be optimal for all applications.

15. Viva Voce Questions

1. Why is logistic regression called a classification algorithm despite the name 'regression'?
2. What is the sigmoid function and why is it used?
3. Define True Positive, True Negative, False Positive, and False Negative.
4. What is the difference between precision and recall?
5. Why is F1-score used instead of accuracy in imbalanced datasets?
6. What does an ROC curve represent?

7. What is AUC and what does a value close to 1 indicate?
8. What is the default decision threshold in logistic regression, and can it be changed?
9. What is the odds ratio in logistic regression?
10. How does class imbalance affect classification metrics?
11. What is the cost function used to train logistic regression?
12. What is the difference between logistic regression and linear regression?
13. What is Maximum Likelihood Estimation?
14. How would you handle a dataset with severe class imbalance?

16. Conclusion

The performance of a logistic regression based binary classifier was successfully illustrated on an open-access dataset using a confusion matrix along with precision, recall, and F1-score, highlighting the strengths and trade-offs of each metric.

Experiment No. 7

Support Vector Machine Classification: Linear and Non-linear Kernels

---

1. Aim / Problem Statement

To utilize Support Vector Machine (SVM) classifiers with linear and non-linear kernels on a suitable open-access dataset and to compare classification performance using appropriate evaluation metrics.

2. Objectives

- To understand the working principle of Support Vector Machines.
- To implement SVM classifiers using linear, polynomial, and RBF (radial basis function) kernels.
- To compare classification performance across kernels using accuracy, precision, recall, and F1-score.
- To visualize decision boundaries for different kernels.

3. Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries: pandas, numpy, scikit-learn (svm.SVC, metrics), matplotlib

4. Introduction

4.1 Concept

Support Vector Machine (SVM) is a supervised learning algorithm that finds the optimal hyperplane which best separates data points of different classes with the maximum possible margin. Points closest to the hyperplane, which influence its position, are called Support Vectors. When data is not linearly separable in its original feature space, SVM uses the "kernel trick" to implicitly map data into a higher-dimensional space where a linear separation becomes possible.

4.2 Background

SVM was developed by Vladimir Vapnik and Alexey Chervonenkis in the 1960s-70s as part of statistical learning theory, with the modern soft-margin and kernel-based formulation introduced by Vapnik, Boser, and Guyon around 1992-1995. It became one of the most popular classification algorithms in the 1990s-2000s, especially for text classification and bioinformatics.

4.3 Applications

- Text and document classification (spam filtering, sentiment analysis).
- Image classification and face detection.

- Bioinformatics: cancer/gene classification.
- Handwriting/digit recognition.

4.4 Advantages

- Effective in high-dimensional spaces, even when features outnumber samples.
- Memory efficient (uses only support vectors).
- Versatile via different kernel functions to model non-linear boundaries.

4.5 Limitations

- Computationally expensive for very large datasets.
- Requires careful tuning of hyperparameters (C, gamma, kernel).
- Does not directly provide probability estimates (requires extra calibration).
- Less effective when classes overlap heavily / noisy data with many overlapping points.

5. Theory

SVM aims to find a hyperplane $w^Tx + b = 0$ that maximises the margin (distance) between the closest points of two classes, subject to correctly classifying (or softly misclassifying, controlled by parameter C) all training points. For non-linearly separable data, a kernel function $K(x_i, x□)$ computes the dot product of the data in a transformed (typically higher-dimensional) feature space, without explicitly performing the transformation — this is the "kernel trick".

5.1 Mathematical Formulas

- Hyperplane: $w^Tx + b = 0$
- Margin: $2 / ||w||$
- Primal objective: minimise $(1/2)||w||^2 + C·\Sigma\xi_i$, subject to $y_i(w^Tx_i+b) \geq 1-\xi_i$
- Linear kernel: $K(x_i,x□) = x_i^Tx□$
- Polynomial kernel: $K(x_i,x□) = (\gamma·x_i^Tx□ + r)^d$
- RBF (Gaussian) kernel: $K(x_i,x□) = \exp(-\gamma·||x_i-x□||^2)$

5.2 Working Principle

The SVM optimizer solves a convex quadratic programming problem (typically via its dual form) to find the support vectors and the corresponding weights that maximise the margin while permitting a controlled number of misclassifications (soft margin, governed by hyperparameter C). For non-linear kernels, the same dual optimisation is performed but using the kernel function K in place of a plain dot product, effectively finding a linear separator in a higher-dimensional implicit feature space.

5.3 Types / Classification

Category	Types
Kernel types	Linear, Polynomial, RBF (Gaussian), Sigmoid
Margin types	Hard Margin SVM, Soft Margin SVM

5.4 Comparison Table

Kernel	Decision Boundary Shape	Key Hyperparameter(s)	Best Suited For
Linear	Straight line / hyperplane	C	Linearly separable data, high-dimensional sparse data (text)
Polynomial	Curved, polynomial boundary	C, degree, gamma	Data with polynomial-like relationships
RBF	Highly flexible, non-linear	C, gamma	Complex, non-linear class boundaries (most general purpose)

Table 7.1: Comparison of techniques.

5.5 Observation Table

S.No.	Kernel	Accuracy	Precision	Recall	F1-score	No. of Support Vectors
1	Linear					
2	Polynomial					
3	RBF					

Table 7.2: Observation table (to be filled during the practical session).

5.6 Figures / Diagrams

Figure 7.1: Decision boundary plots for Linear, Polynomial, and RBF kernels on a 2-feature projection of the dataset. Figure 7.2: Bar chart comparing accuracy across kernels — insert after execution.

Reference link(s): https://scikit-learn.org/stable/auto_examples/svm/plot_svm_kernels.html for reference SVM kernel decision-boundary diagrams.

6. Algorithm

1. Start.
2. Import libraries and load an open-access classification dataset (e.g., Iris or Breast Cancer dataset).
3. Split data into features (X) and target (y); split into training and testing sets.
4. Standardize features (important for SVM, which is distance-based).
5. Train SVC with kernel='linear'; predict and evaluate.
6. Train SVC with kernel='poly'; predict and evaluate.
7. Train SVC with kernel='rbf'; predict and evaluate.

8. Compute accuracy, precision, recall, F1-score for each kernel.
9. Plot decision boundaries (using two selected/PCA-reduced features) for visual comparison.
10. Compare and interpret results across kernels.
11. Stop.

7. Flowchart

Draw a flowchart representing the algorithm above (Start → Load Data → Pre-process → Fit Model → Predict → Evaluate → Visualize → Stop). A ready-reference flowchart / diagram for this technique can be found at the link below:

Reference link(s): https://scikit-learn.org/stable/auto_examples/svm/plot_svm_kernels.html for reference SVM kernel decision-boundary diagrams.

8. Program / Source Code (Python – Jupyter Notebook / Google Colab)

```
# Experiment 7: SVM with Linear and Non-linear Kernels

import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score


data = load_breast_cancer()

X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,

                                        random_state=42, stratify=y)

scaler = StandardScaler()

X_train_s = scaler.fit_transform(X_train)

X_test_s = scaler.transform(X_test)


kernels = ['linear', 'poly', 'rbf']
```

```
results = {}
for k in kernels:
    clf = SVC(kernel=k, C=1.0, gamma='scale', degree=3)
    clf.fit(X_train_s, y_train)
    y_pred = clf.predict(X_test_s)
    results[k] = {
        'accuracy': accuracy_score(y_test, y_pred),
        'precision': precision_score(y_test, y_pred),
        'recall': recall_score(y_test, y_pred),
        'f1': f1_score(y_test, y_pred),
        'n_support_vectors': clf.support_vectors_.shape[0]
    }
    print(k, results[k])

# Decision boundary visualization using first two features
from sklearn.svm import SVC as SVC2
X2 = X_train_s[:, :2]
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, k in zip(axes, kernels):
    clf2 = SVC2(kernel=k, C=1.0, gamma='scale').fit(X2, y_train)
    xx, yy = np.meshgrid(np.linspace(X2[:,0].min()-1, X2[:,0].max()+1, 200),
                         np.linspace(X2[:,1].min()-1, X2[:,1].max()+1, 200))
    Z = clf2.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    ax.contourf(xx, yy, Z, alpha=0.3)
    ax.scatter(X2[:,0], X2[:,1], c=y_train, edgecolors='k', s=15)
    ax.set_title(f'SVM - {k} kernel')
plt.tight_layout(); plt.show()
```

9. Sample Input

Breast Cancer Wisconsin dataset (569 samples, 30 features, binary target), standardized before use.

10. Expected Output

Accuracy, precision, recall, F1-score for each kernel (typically Linear/RBF ≈ 0.96-0.98, Polynomial slightly lower/variable); decision boundary plots for each kernel on a 2-feature projection.

11. Result

SVM classifiers with linear, polynomial, and RBF kernels were successfully implemented and their classification performance compared using standard evaluation metrics.

12. Applications

- Text/document classification.
- Image and face recognition.
- Bioinformatics (cancer classification, gene expression analysis).
- Handwritten digit recognition.

13. Advantages

- Effective in high-dimensional spaces.
- Kernel trick allows modelling complex non-linear boundaries.
- Robust to overfitting in high-dimensional space when properly regularized (C).

14. Disadvantages

- Computationally expensive on large datasets.
- Requires careful hyperparameter tuning (C, gamma, kernel, degree).
- Less interpretable than logistic regression or decision trees.

15. Viva Voce Questions

1. What is a Support Vector Machine?
2. What are support vectors?
3. What is the margin in SVM, and why is it maximised?
4. What is the kernel trick and why is it useful?
5. Differentiate between hard margin and soft margin SVM.
6. What is the role of the hyperparameter C in SVM?
7. What is the role of gamma in the RBF kernel?
8. When would you prefer a linear kernel over an RBF kernel?
9. Why is feature scaling important for SVM?
10. What is the difference between SVM and logistic regression?
11. How does SVM handle multi-class classification?

12. What is overfitting in the context of SVM, and how can it be controlled?
13. What are the disadvantages of SVM on large datasets?
14. Explain the polynomial kernel formula and its parameters.

16. Conclusion

SVM classifiers using linear and non-linear (polynomial, RBF) kernels were successfully applied to an open-access dataset, showing that kernel choice significantly influences the shape of the decision boundary and overall classification performance.

Experiment No. 8

K-Means Clustering with Optimal Cluster Selection using Elbow / Silhouette Method

---

1. Aim / Problem Statement

To apply the K-Means clustering algorithm on a suitable open-access dataset and to obtain the optimal number of clusters using the Elbow method or the Silhouette method.

2. Objectives

- To understand K-Means as an unsupervised, centroid-based clustering algorithm.
- To implement K-Means clustering on an open-access dataset.
- To determine the optimal number of clusters (k) using the Elbow method (WCSS) and the Silhouette method.
- To visualize and interpret the resulting clusters.

3. Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries: pandas, numpy, scikit-learn (cluster.KMeans, metrics.silhouette_score), matplotlib, seaborn

4. Introduction

4.1 Concept

K-Means is an unsupervised machine learning algorithm used to partition a dataset into k distinct, non-overlapping clusters based on feature similarity, without using any labelled output. Each cluster is represented by its centroid (the mean of all points assigned to it), and the algorithm iteratively assigns points to the nearest centroid and updates centroids until convergence.

4.2 Background

The K-Means algorithm was first proposed by Hugo Steinhaus (1957) and later refined into the standard iterative algorithm by Stuart Lloyd (1957, published 1982) and James MacQueen (1967), who coined the term "k-means". It remains one of the simplest and most widely used clustering algorithms due to its speed and scalability.

4.3 Applications

- Customer segmentation in marketing.
- Image compression / colour quantisation.
- Document/topic clustering.
- Anomaly detection (points far from any cluster centroid).

4.4 Advantages

- Simple, fast, and scalable to large datasets.
- Easy to interpret and implement.
- Guaranteed to converge (to a local optimum).

4.5 Limitations

- Requires the number of clusters (k) to be specified in advance.
- Sensitive to initial centroid placement (mitigated by k-means++).
- Assumes roughly spherical, equally-sized clusters; struggles with non-convex cluster shapes.
- Sensitive to outliers and feature scaling.

5. Theory

K-Means minimises the Within-Cluster Sum of Squares (WCSS), also called inertia — the total squared distance between each point and its assigned cluster centroid. The algorithm proceeds iteratively: (1) initialise k centroids (commonly via the k-means++ strategy for better initial spread), (2) assign each data point to its nearest centroid, (3) recompute each centroid as the mean of the points assigned to it, and (4) repeat steps 2-3 until centroids no longer change significantly (convergence) or a maximum number of iterations is reached.

Since k must be chosen in advance, two standard techniques are used to determine the optimal k: the Elbow Method, which plots WCSS against k and looks for the point where the rate of decrease sharply slows (the "elbow"), and the Silhouette Method, which measures how similar a point is to its own cluster compared to other clusters.

5.1 Mathematical Formulas

- WCSS (Inertia): $WCSS = \Sigma_{} \Sigma_{x \in C_{}} \|x - \mu_{}\|^2$
- Centroid update: $\mu_{} = (1/|C_{}|) \Sigma_{x \in C_{}} x$
- Silhouette score for point i: $s(i) = (b(i) - a(i)) / \max(a(i), b(i))$
- where a(i) = mean intra-cluster distance, b(i) = mean nearest-cluster distance
- Overall Silhouette Score = mean of s(i) over all points (range: −1 to +1)

5.2 Working Principle

K-Means alternates between an assignment step (Expectation) — assigning each point to its nearest centroid using Euclidean distance — and an update step (Maximisation) — recomputing centroids as the mean of assigned points. This is analogous to a simplified Expectation-Maximisation (EM) procedure and converges when cluster assignments stabilise.

5.3 Types / Classification

Category	Types
K-Means variants	Standard K-Means, K-Means++ (smart initialization), Mini-Batch K-Means (for large data), K-Medoids (robust to outliers)

Cluster validation methods	Elbow Method (WCSS), Silhouette Method, Davies-Bouldin Index, Calinski-Harabasz Index

5.4 Comparison Table

Method	What it Measures	How Optimal k is Chosen
Elbow Method	WCSS (inertia) vs k	Point where WCSS decrease sharply flattens ('elbow')
Silhouette Method	Cluster cohesion vs separation	k with maximum average silhouette score

Table 8.1: Comparison of techniques.

5.5 Observation Table

k	WCSS (Inertia)	Silhouette Score
2		
3		
4		
5		
6		

Table 8.2: Observation table (to be filled during the practical session).

5.6 Figures / Diagrams

Figure 8.1: Elbow plot (WCSS vs k). Figure 8.2: Silhouette score plot vs k. Figure 8.3: Scatter plot of final clusters with centroids marked — insert after execution.

Reference link(s):
https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html for a reference silhouette analysis diagram, and https://scikit-learn.org/stable/modules/clustering.html#k-means for the K-Means algorithm diagram.

6. Algorithm

1. Start.
2. Import libraries and load a suitable open-access dataset (e.g., Mall Customer Segmentation dataset or Iris dataset).
3. Select relevant numeric features and standardize them.
4. For k = 2 to 10: fit KMeans(n_clusters=k) and record WCSS (inertia_) and silhouette_score.
5. Plot WCSS vs k (Elbow plot) and identify the elbow point.
6. Plot Silhouette score vs k and identify the k with maximum score.
7. Select the optimal k based on both methods.

8. Fit the final KMeans model with the chosen optimal k.
9. Visualize the resulting clusters (scatter plot with centroids).
10. Interpret the characteristics of each cluster.
11. Stop.

7. Flowchart

Draw a flowchart representing the algorithm above (Start → Load Data → Pre-process → Fit Model → Predict → Evaluate → Visualize → Stop). A ready-reference flowchart / diagram for this technique can be found at the link below:

Reference link(s):
https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html for a reference silhouette analysis diagram, and https://scikit-learn.org/stable/modules/clustering.html#k-means for the K-Means algorithm diagram.

8. Program / Source Code (Python – Jupyter Notebook / Google Colab)

```
# Experiment 8: K-Means Clustering with Elbow and Silhouette Method
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

data = load_iris()
X = data.data
X_scaled = StandardScaler().fit_transform(X)

wcss, sil_scores = [], []
K_range = range(2, 11)
for k in K_range:
    km = KMeans(n_clusters=k, init='k-means++', random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
```

```
    wcss.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, labels))
    print(f"k={k}: WCSS={km.inertia_:.2f}, Silhouette={sil_scores[-1]:.4f}")

# Elbow plot
plt.plot(list(K_range), wcss, marker='o')
plt.xlabel('Number of Clusters (k)'); plt.ylabel('WCSS')
plt.title('Elbow Method'); plt.show()

# Silhouette plot
plt.plot(list(K_range), sil_scores, marker='o', color='green')
plt.xlabel('Number of Clusters (k)'); plt.ylabel('Silhouette Score')
plt.title('Silhouette Method'); plt.show()

# Final model with optimal k (say k=3 for Iris)
optimal_k = 3
final_km = KMeans(n_clusters=optimal_k, init='k-means++', random_state=42, n_init=10)
final_labels = final_km.fit_predict(X_scaled)

plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=final_labels, cmap='viridis', s=30)
plt.scatter(final_km.cluster_centers_[:, 0], final_km.cluster_centers_[:, 1],
            c='red', marker='X', s=200, label='Centroids')
plt.legend(); plt.title(f'K-Means Clustering (k={optimal_k})'); plt.show()
```

9. Sample Input

Iris dataset (150 samples, 4 numeric features: sepal length/width, petal length/width), standardized before clustering.

10. Expected Output

WCSS and silhouette score table for k=2 to 10; elbow plot showing a bend around k=3; silhouette plot peaking around k=2-3; final scatter plot showing 3 clusters with centroids.

11. Result

The K-Means clustering algorithm was successfully applied and the optimal number of clusters was determined as k=3 using both the Elbow and Silhouette methods, consistent with the three known Iris species.

12. Applications

- Customer segmentation for targeted marketing.
- Image compression/colour quantisation.
- Market basket / product grouping.
- Document and topic clustering.

13. Advantages

- Simple, fast, and scalable to large datasets.
- Easy to interpret cluster centroids.
- Works well when clusters are roughly spherical and well-separated.

14. Disadvantages

- Requires k to be pre-specified.
- Sensitive to outliers and initial centroid placement.
- Poor performance on non-convex or unevenly sized/dense clusters.

15. Viva Voce Questions

1. What is K-Means clustering and what type of learning is it?
2. What does WCSS/inertia measure?
3. Explain the Elbow method for choosing k.
4. Explain the Silhouette method and the range of the silhouette score.
5. Why is k-means++ initialization preferred over random initialization?
6. What are the limitations of K-Means clustering?
7. Why is feature scaling important before applying K-Means?
8. How does K-Means handle outliers?
9. What is the difference between K-Means and hierarchical clustering?
10. What distance metric does standard K-Means use?
11. Can K-Means clustering results change on different runs? Why?
12. What is Mini-Batch K-Means and when would you use it?
13. What does a silhouette score close to 0 or negative indicate?
14. How would you cluster data with non-spherical cluster shapes?

16. Conclusion

K-Means clustering was successfully implemented on an open-access dataset, and the optimal number of clusters was determined using both the Elbow method and Silhouette method, demonstrating how internal validation techniques guide unsupervised model selection.

Experiment No. 9

Density-Based Clustering using DBSCAN

---

1. Aim / Problem Statement

To employ a density-based clustering algorithm such as DBSCAN using a suitable open-access dataset to identify the clustering structure and noise points.

2. Objectives

- To understand the concept of density-based clustering.
- To implement the DBSCAN algorithm on an open-access dataset.
- To identify core points, border points, and noise points.
- To tune DBSCAN hyperparameters (eps and min_samples) and compare with K-Means clustering.

3. Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries: pandas, numpy, scikit-learn (cluster.DBSCAN, neighbors.NearestNeighbors), matplotlib

4. Introduction

4.1 Concept

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is an unsupervised clustering algorithm that groups together points that are closely packed (high density regions), while marking points in low-density regions as outliers/noise. Unlike K-Means, DBSCAN does not require the number of clusters to be specified in advance and can discover clusters of arbitrary shape.

4.2 Background

DBSCAN was proposed by Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu in 1996. It received the SIGKDD Test of Time Award in 2014 for its lasting impact and continues to be a standard density-based clustering algorithm alongside its successors such as OPTICS and HDBSCAN.

4.3 Applications

- Anomaly/outlier detection in network intrusion detection systems.
- Geospatial data analysis (e.g., clustering GPS locations, identifying hotspots).
- Image segmentation.
- Identifying arbitrary-shaped clusters in scientific data (e.g., astronomy, seismology).

4.4 Advantages

- Does not require the number of clusters to be specified beforehand.
- Can find arbitrarily shaped clusters (not just spherical).
- Naturally identifies and separates out noise/outlier points.

4.5 Limitations

- Sensitive to the choice of eps and min_samples hyperparameters.
- Struggles with clusters of varying density.
- Not well-suited for very high-dimensional data (distance becomes less meaningful).

5. Theory

DBSCAN classifies each point in the dataset into one of three categories based on two parameters: eps (ε, the neighbourhood radius) and min_samples (minimum number of points required to form a dense region):

• Core Point: a point that has at least min_samples points (including itself) within distance eps.

• Border Point: a point that is within eps distance of a core point but does not itself have min_samples neighbours.

• Noise Point: a point that is neither a core point nor a border point (i.e., an outlier).

The algorithm starts with an arbitrary unvisited point, retrieves its eps-neighbourhood; if it contains at least min_samples points, a new cluster is formed and expanded by recursively including density-reachable points. If not, the point is temporarily labelled as noise (it may later be claimed as a border point of another cluster).

5.1 Mathematical Formulas

- Eps-neighbourhood of point p: $N_{e}(p) = \{q \in D \mid dist(p,q) \leq eps\}$
- Core point condition: $|N_{e}(p)| \geq min_samples$
- Density-reachability: q is density-reachable from p if there is a chain $p=p_1,p_2,...,p_n=q$ where each $p_{i+1} \in N_{e}(p_i)$ and p_i is a core point
- Distance metric (typical): Euclidean distance $dist(p,q) = \sqrt{\Sigma(p_i-q_i)^2}$

5.2 Working Principle

DBSCAN grows clusters by starting from a core point and iteratively adding all points that are density-reachable from it, expanding the cluster boundary outward through connected dense regions. Points that cannot be density-connected to any cluster remain labelled as noise (label = −1 in scikit-learn's implementation). The k-distance graph (sorting distances to the k-th nearest neighbour) is commonly used to help choose an appropriate eps value.

5.3 Types / Classification

Category	Types
Related density-based algorithms	DBSCAN, OPTICS (handles varying density), HDBSCAN (hierarchical density-based)
Point categories in DBSCAN	Core Point, Border Point, Noise Point

5.4 Comparison Table

Aspect	K-Means	DBSCAN
Number of clusters	Must be specified (k)	Determined automatically
Cluster shape	Spherical/convex	Arbitrary shape
Handles noise/outliers?	No (every point assigned to a cluster)	Yes (explicitly labels noise as -1)
Sensitive to	Initial centroids, k	eps, min_samples

Table 9.1: Comparison of techniques.

5.5 Observation Table

S.No.	eps	min_samples	No. of Clusters Found	No. of Noise Points
1	0.3	5		
2	0.5	5		
3	0.5	10		

Table 9.2: Observation table (to be filled during the practical session).

5.6 Figures / Diagrams

Figure 9.1: k-distance graph used to select eps (sorted distances to k-th nearest neighbour, look for the 'knee'). Figure 9.2: Scatter plot of DBSCAN clusters with noise points marked distinctly (e.g., in black) — insert after execution.

Reference link(s): https://scikit-learn.org/stable/auto_examples/cluster/plot_dbscan.html for a reference DBSCAN clustering diagram and comparison plot.

6. Algorithm

1. Start.
2. Import libraries and load/generate a suitable open-access dataset with non-spherical structure (e.g., sklearn.datasets.make_moons, or a real open-access spatial dataset).
3. Standardize the features.
4. Use NearestNeighbors to compute the k-distance graph and plot it to help select an appropriate eps.
5. Initialize DBSCAN with chosen eps and min_samples values.

6. Fit DBSCAN on the dataset and obtain cluster labels (with -1 denoting noise).
7. Count the number of clusters formed and the number of noise points.
8. Visualize the clustering result, plotting noise points distinctly.
9. Experiment with different eps/min_samples values and observe the effect on clustering.
10. Compare results with K-Means clustering on the same dataset.
11. Stop.

7. Flowchart

Draw a flowchart representing the algorithm above (Start → Load Data → Pre-process → Fit Model → Predict → Evaluate → Visualize → Stop). A ready-reference flowchart / diagram for this technique can be found at the link below:

Reference link(s): https://scikit-learn.org/stable/auto_examples/cluster/plot_dbscan.html for a reference DBSCAN clustering diagram and comparison plot.

8. Program / Source Code (Python – Jupyter Notebook / Google Colab)

```
# Experiment 9: DBSCAN Clustering

import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import make_moons

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import DBSCAN, KMeans

from sklearn.neighbors import NearestNeighbors


# 1. Generate/Load a dataset with non-spherical structure

X, _ = make_moons(n_samples=300, noise=0.08, random_state=42)

X_scaled = StandardScaler().fit_transform(X)


# 2. k-distance graph to help choose eps

neighbors = NearestNeighbors(n_neighbors=5).fit(X_scaled)

distances, _ = neighbors.kneighbors(X_scaled)

distances = np.sort(distances[:, 4])

plt.plot(distances)
```

```
plt.xlabel('Points sorted by distance'); plt.ylabel('5th Nearest Neighbour Distance')
plt.title('k-distance Graph (for eps selection)'); plt.show()

# 3. Apply DBSCAN
db = DBSCAN(eps=0.25, min_samples=5)
labels = db.fit_predict(X_scaled)

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = list(labels).count(-1)
print(f"Estimated number of clusters: {n_clusters}")
print(f"Estimated number of noise points: {n_noise}")

# 4. Visualize DBSCAN clusters
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap='plasma', s=30)
plt.title('DBSCAN Clustering (noise = label -1)'); plt.show()

# 5. Compare with K-Means on the same data
km = KMeans(n_clusters=2, random_state=42, n_init=10).fit(X_scaled)
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=km.labels_, cmap='viridis', s=30)
plt.title('K-Means Clustering (for comparison)'); plt.show()
```

9. Sample Input

Two-moons synthetic dataset (300 samples, 2 features, non-convex crescent-shaped clusters) generated via sklearn.datasets.make_moons, standardized before use.

10. Expected Output

k-distance graph for eps selection; DBSCAN correctly identifies 2 crescent-shaped clusters with few/no noise points, while K-Means incorrectly splits the crescents due to its spherical-cluster assumption.

11. Result

DBSCAN was successfully applied to identify arbitrarily shaped clusters and noise points, clearly outperforming K-Means on non-convex cluster structures.

12. Applications

- Anomaly/fraud/intrusion detection.
- Geospatial hotspot analysis.
- Image segmentation.
- Discovering clusters of arbitrary shape in scientific datasets.

13. Advantages

- No need to pre-specify number of clusters.
- Detects arbitrarily shaped clusters.
- Robust to outliers by explicitly labelling them as noise.

14. Disadvantages

- Highly sensitive to eps and min_samples selection.
- Performs poorly when clusters have widely varying densities.
- Less effective in high-dimensional feature spaces.

15. Viva Voce Questions

1. What does DBSCAN stand for?
2. Differentiate between core points, border points, and noise points.
3. What are the two key hyperparameters of DBSCAN?
4. How is the eps parameter typically chosen?
5. Why doesn't DBSCAN require the number of clusters to be specified?
6. What is a major advantage of DBSCAN over K-Means?
7. What is a major limitation of DBSCAN?
8. What does a label of -1 mean in scikit-learn's DBSCAN output?
9. Why does DBSCAN struggle with clusters of varying density?
10. What is density-reachability?
11. What is the k-distance graph used for?
12. How does DBSCAN handle outliers compared to K-Means?
13. What is OPTICS and how does it improve upon DBSCAN?
14. Can DBSCAN be used on high-dimensional data effectively? Why or why not?

16. Conclusion

The DBSCAN density-based clustering algorithm was successfully employed on an open-access dataset, correctly identifying arbitrarily shaped clusters and separating out noise points, demonstrating its advantage over centroid-based methods like K-Means for non-convex cluster structures.

Experiment No. 10

Ensemble Learning: Random Forest / Boosting vs Baseline Model

---

1. Aim / Problem Statement

To analyze the performance of ensemble learning techniques such as Random Forest or Boosting on a suitable open-access dataset in comparison with a baseline machine learning model.

2. Objectives

- To understand the concept of ensemble learning (Bagging and Boosting).
- To implement a Random Forest classifier and a Boosting algorithm (e.g., AdaBoost/Gradient Boosting).
- To compare their performance against a baseline single model (e.g., a single Decision Tree).
- To analyze feature importance derived from ensemble models.

3. Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries: pandas, numpy, scikit-learn (ensemble.RandomForestClassifier, ensemble.GradientBoostingClassifier / AdaBoostClassifier, tree.DecisionTreeClassifier), matplotlib

4. Introduction

4.1 Concept

Ensemble learning combines predictions from multiple individual ("weak" or "base") models to produce a single, more accurate and robust prediction than any individual model alone. The two dominant ensemble strategies are Bagging (Bootstrap Aggregating), which trains many models independently on random subsets of data and averages/votes their predictions (e.g., Random Forest), and Boosting, which trains models sequentially, with each new model focusing on correcting the errors of the previous ones (e.g., AdaBoost, Gradient Boosting).

4.2 Background

Bagging was introduced by Leo Breiman (1996), who also proposed the Random Forest algorithm (2001), which adds random feature selection on top of bagged decision trees to further decorrelate the trees. Boosting originated with Freund and Schapire's AdaBoost algorithm (1995/1997), later generalised into Gradient Boosting by Jerome Friedman (2001), which underlies modern implementations such as XGBoost, LightGBM, and CatBoost.

4.3 Applications

- Credit scoring and fraud detection.

- Medical diagnosis and prognosis prediction.
- Search-engine ranking (Gradient Boosted Trees).
- Kaggle-style structured/tabular-data competitions, where ensembles routinely outperform single models.

4.4 Advantages

- Substantially reduces variance (Bagging/Random Forest) and/or bias (Boosting) compared to a single model.
- Generally more accurate and robust to overfitting than individual base learners.
- Provides built-in feature importance estimates.

4.5 Limitations

- Less interpretable than a single decision tree (a "black box" of many trees).
- Computationally more expensive to train and predict than a single model.
- Boosting algorithms can overfit if the number of estimators/learning rate is not tuned carefully.

5. Theory

Random Forest (Bagging-based): Builds a large number of decision trees, each trained on a bootstrap sample (random sample with replacement) of the training data, and at each split, considers only a random subset of features. Final predictions are obtained by majority voting (classification) or averaging (regression) across all trees, which reduces variance and overfitting compared to a single deep decision tree.

Boosting (e.g., AdaBoost / Gradient Boosting): Builds an ensemble of weak learners (typically shallow decision trees/"stumps") sequentially. AdaBoost increases the weights of misclassified samples so subsequent learners focus more on "hard" cases, and combines all learners via a weighted vote. Gradient Boosting instead fits each new weak learner to the residual errors (negative gradient of the loss function) of the combined ensemble so far, progressively reducing the overall prediction error.

5.1 Mathematical Formulas

- Random Forest prediction (classification): $\hat{y} = \text{mode}\{ h_1(x), h_2(x), ..., h_T(x) \}$ (majority vote of T trees)
- Bagging variance reduction (for i.i.d. base learners with variance σ^2): $\text{Var(average)} = \sigma^2/T$
- AdaBoost sample weight update: $w_i \leftarrow w_i \cdot \exp(\alpha \cdot \mathbb{1}[y_i \neq h_t(x_i)])$
- AdaBoost learner weight: $\alpha_t = 0.5 \cdot \ln((1-\varepsilon_t)/\varepsilon_t)$ (ε_t = weighted error rate of learner t)
- Gradient Boosting update: $F_t(x) = F_\{m-1\}(x) + \nu \cdot h_m(x)$ (ν = learning rate, h_m fits residuals)

5.2 Working Principle

Random Forest reduces variance by averaging many de-correlated trees (achieved via bootstrap sampling + random feature subsets), following the principle that the variance of an

average of T independent estimators is 1/T times the variance of a single estimator. Boosting reduces bias by additively combining weak learners, where each new learner is trained to correct the residual errors made by the ensemble built so far, controlled by a learning rate to avoid overfitting.

5.3 Types / Classification

Category	Types
Bagging-based ensembles	Random Forest, Bagged Decision Trees, Extra Trees
Boosting-based ensembles	AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost
Other ensemble strategies	Stacking, Voting Classifier/Regressor

5.4 Comparison Table

Aspect	Bagging (Random Forest)	Boosting (AdaBoost/GBM)
Training	Parallel, independent trees	Sequential, each tree corrects previous errors
Primary goal	Reduce variance	Reduce bias (and variance)
Overfitting risk	Lower (more robust)	Higher (needs careful tuning of n_estimators/learning_rate)
Typical base learner	Deep/full decision trees	Shallow trees ("stumps")

Table 10.1: Comparison of techniques.

5.5 Observation Table

S.No.	Model	Accuracy	Precision	Recall	F1-score	Training Time (sec)
1	Decision Tree (baseline)					
2	Random Forest					
3	Gradient Boosting / AdaBoost					

Table 10.2: Observation table (to be filled during the practical session).

5.6 Figures / Diagrams

Figure 10.1: Bar chart comparing accuracy/F1-score of Decision Tree vs Random Forest vs Boosting. Figure 10.2: Feature importance bar chart from the Random Forest model — insert after execution.

Reference link(s): https://scikit-learn.org/stable/modules/ensemble.html#random-forests and https://scikit-learn.org/stable/modules/ensemble.html#gradient-boosting for reference ensemble-learning architecture diagrams.

6. Algorithm

1. Start.
2. Import libraries and load an open-access classification dataset (e.g., Breast Cancer Wisconsin dataset).
3. Split data into features (X) and target (y); split into training and testing sets.
4. Train a baseline Decision Tree classifier; predict and evaluate on the test set.
5. Train a Random Forest classifier (e.g., n_estimators=100); predict and evaluate.
6. Train a Boosting classifier (e.g., GradientBoostingClassifier or AdaBoostClassifier); predict and evaluate.
7. Compute accuracy, precision, recall, F1-score, and training time for each model.
8. Extract and plot feature importances from the Random Forest model.
9. Compare all three models in a summary table/chart.
10. Interpret which ensemble technique performs best and why.
11. Stop.

7. Flowchart

Draw a flowchart representing the algorithm above (Start → Load Data → Pre-process → Fit Model → Predict → Evaluate → Visualize → Stop). A ready-reference flowchart / diagram for this technique can be found at the link below:

Reference link(s): https://scikit-learn.org/stable/modules/ensemble.html#random-forests and https://scikit-learn.org/stable/modules/ensemble.html#gradient-boosting for reference ensemble-learning architecture diagrams.

8. Program / Source Code (Python – Jupyter Notebook / Google Colab)

```
# Experiment 10: Ensemble Learning - Random Forest & Boosting vs Baseline

import time

import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score


data = load_breast_cancer()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
                                        random_state=42, stratify=y)


models = {
   "Decision Tree (baseline)": DecisionTreeClassifier(random_state=42),
   "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
   "Gradient Boosting": GradientBoostingClassifier(n_estimators=100, random_state=42)
}


results = {}
for name, model in models.items():
   t0 = time.time()
   model.fit(X_train, y_train)
   t1 = time.time()
   y_pred = model.predict(X_test)
   results[name] = {
      "Accuracy": accuracy_score(y_test, y_pred),
      "Precision": precision_score(y_test, y_pred),
      "Recall": recall_score(y_test, y_pred),
      "F1": f1_score(y_test, y_pred),
      "Time(s)": t1 - t0
   }
   print(name, results[name])
```

```
# Feature importance from Random Forest
rf = models["Random Forest"]
importances = rf.feature_importances_
idx = np.argsort(importances)[-10:]
plt.barh(range(len(idx)), importances[idx])
plt.yticks(range(len(idx)), [data.feature_names[i] for i in idx])
plt.xlabel('Feature Importance'); plt.title('Top 10 Important Features - Random Forest')
plt.tight_layout(); plt.show()
```

9. Sample Input

Breast Cancer Wisconsin dataset (569 samples, 30 numeric features, binary target: malignant/benign).

10. Expected Output

Comparison table of Accuracy, Precision, Recall, F1-score, and training time for Decision Tree, Random Forest, and Gradient Boosting (ensemble methods typically achieve higher/more stable accuracy, e.g., ~0.96-0.98, than the single Decision Tree baseline); feature importance bar chart from Random Forest.

11. Result

Ensemble learning techniques (Random Forest and Gradient Boosting) were successfully implemented and shown to outperform the baseline single Decision Tree model in terms of accuracy and robustness.

12. Applications

- Credit scoring and fraud detection.
- Medical diagnosis and risk prediction.
- Search ranking and recommendation systems.
- General-purpose tabular data competitions and industry ML pipelines.

13. Advantages

- Higher accuracy and robustness than single models.
- Reduced overfitting (Random Forest) via averaging of de-correlated trees.
- Built-in feature importance ranking aids interpretability at the ensemble level.

14. Disadvantages

- Less interpretable ('black box') than a single decision tree.

- Higher computational and memory cost, especially for large forests/many boosting rounds.
- Boosting models require careful hyperparameter tuning to avoid overfitting.

15. Viva Voce Questions

1. What is ensemble learning?
2. Differentiate between Bagging and Boosting.
3. How does a Random Forest reduce overfitting compared to a single Decision Tree?
4. What is bootstrap sampling?
5. Explain how AdaBoost updates sample weights.
6. What is the difference between AdaBoost and Gradient Boosting?
7. What is the role of the learning rate in Gradient Boosting?
8. How is feature importance computed in a Random Forest?
9. Why can Boosting be more prone to overfitting than Bagging?
10. What is the difference between a weak learner and a strong learner?
11. What are out-of-bag (OOB) samples in Random Forest?
12. How does the number of estimators (n_estimators) affect Random Forest performance?
13. What is stacking, and how does it differ from bagging/boosting?
14. Why do ensemble methods generally outperform individual base models?

16. Conclusion

Ensemble learning techniques — Random Forest (bagging) and Gradient Boosting — were successfully analyzed on an open-access dataset and shown to outperform a baseline single Decision Tree model, illustrating how combining multiple weak/base learners improves predictive accuracy and robustness in machine learning.